به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین دوم

مدرس: دکتر محمد قدسی، مهدی صفرنژاد — انتشار: ۲۰ اسفند ۹۹

مرتب‌سازی، بازگشتی و داده‌ساختارهای پایه

زمان تحویل این تمرین ۱۵ فروردین ۱۴۰۰ می‌باشد. سؤالات را با دقت بخوانید و روی همه آن‌ها وقت بگذارید. تمرین‌های تئوری تحویل گرفته نمی‌شوند اما از آن‌ها سؤالات کوییز مشخص می‌شود. بنابراین روی سؤالات به خوبی فکر کنید و در کلاس‌های حل تمرین مربوطه شرکت کنید.

سؤال ۱. با استفاده از داده ساختار پشته به سوالات زیر پاسخ دهید.

آ. با استفاده از پشته، داده ساختار صف را پیاده‌سازی کنید به طوری که هزینه عملیات اضافه کردن به صف در آن از $O(1)$ باشد. هزینه سرشکن n عمل اضافه و حذف از صف را با استفاد از روش تابع پتانسیل تحلیل کنید. فرض کنید هزینه ها معادل تعداد $push$ها و popها است.

ب. به کمک پشته و حافظه اضافی از مرتبه $O(1)$ یک داده ساختار جدید مانند پشته ارائه دهید که بتواند علاوه بر عملیات‌های $push$ و pop، عملیات‌های $find_min$ و $find_max$ را از مرتبه زمانی $O(1)$ انجام دهد.

پ. آیا می‌توان داده ساختار قسمت ب را به گونه‌ای تغییر دهیم که عملیات $delete_min$ را نیز در $O(1)$ انجام دهد؟

پاسخ: می‌دانیم که پشته‌ها FILO و صف‌ها FIFO هستند. حال فرض کنید که دو پشته داریم که پشته‌ی اول پر و پشته‌ی دوم خالی می‌باشد. حال به ترتیب اجزای پشته‌ی اول را خوانده و در پشته‌ی دوم می‌ریزیم، چون پشته‌ها FILO هستند، اجزایی که زودتر وارد پشته‌ی اول شده‌اند دیرتر وارد پشته‌ی دوم می‌شوند. چون پشته‌ی دوم نیز FILO است، اجزایی که دیرتر وارد شده‌اند زودتر خارج می‌شوند، پس هر عضوی که زودتر وارد پشته‌ی اول شده باشد، بعد از خالی کردن پشته‌ی اول به پشته‌ی دوم زودتر خارج می‌شود که مانند یک FIFO می باشد. حال با کمک این ایده به کمک دو پشته یک صف را طراحی می‌کنیم:

هنگام push به پشته‌ی اول push می‌کنیم. هنگام pop نیز از پشته‌ی دوم pop میکنیم، اما اگر پشته‌ی دوم خالی بود، ابتدا تمام اعضای پشته‌ی اول را وارد پشته‌ی دوم می‌کنیم. سپس از پشته‌ی دوم pop میکنیم. عملیات push چون عملیات push به پشته است از $O(1)$ است. عملیات pop شامل حداکثر n عمل pop برای خالی کردن پشته‌ی اول n عمل push برای وارد شدن به پشته‌ی دوم و یک عملیات pop از پشته‌ی دوم می‌باشد. پس از $O(n)$ می‌باشد. (گر چه پس از یادگیری تحلیل سرشکن می‌توانید بررسی کنید که مرتبه‌ی زمانی pop نیز به طور سرشکن از $O(1)$ می‌باشد چون هر عضو حداکثر دو بار pop می‌شود.)

برای محاسبه هزینه سرشکن:

$$\phi = 2len(S_A)$$

$$\hat{c}_i = c_i + \phi_i - \phi_{i-1}$$

چون در این سوال هزینه ی push و pop یکسان است پس داریم :

$$c_{ipop} = c_{ipush}$$

برای Enqueue فقط یک push در استک A داریم که c_i در آن یک است که به علاوه ی دوبرابر تفاوت فاصله ی استک A قبل و بعد از push کردن می‌شود.

$$Enqueue \rightarrow \hat{c}_i = 1 + 2(len(S_A) - len(S_{A-1})) = 1 + 2 = 3 = O(1)$$

برای Dequeue ابتدا حالتی را در نظر میگیریم که استک B خالی نباشد. در این صورت تنها کافی است یکی از B برداریم.

$$Dequeue \rightarrow if len(S_B) > 0 \rightarrow \hat{c}_i = 1 + 2(len(S_A) - len(S_{A-1})) = 1 + 0 = 1 = O(1)$$

در حالتی که استک B خالی است باید تمام استک A را پاپ کنیم و بعد به اندازه ی طول A در استک B بریزیم و در آخر یکی pop می‌کنیم.

$$c_i = 2len(S_A) + 1$$

$$\phi_i = 0$$

$$\phi_{i-1} = 2len(S_A)$$

بنابراین $\hat{c}_i = 1 = O(1)$

بنابراین هزینه سرشکن برای این پیاده سازی برای Enqueue برابر ۳ است و برای Dequeue برابر ۱ است.

قسمت ب و پ: داده ساختار اصلی یک پشته برای عملیات‌های push و pop است. برای یافتن min نیز از یک پشته‌ی کمکی هم ارتفاع با پشته‌ی اصلی استفاده می‌کنیم که در ارتفاع hم آن، کمینه‌ی h عنصر اول پشته‌ی اصلی را نگه می‌داریم. برای مدیریت این استک نیز به صورت زیر عمل می‌کنیم:

هنگام pop از استک اصلی، از این استک نیز pop می‌کنیم. هنگام push کردن به استک اصلی، عضو جدید را با اخرین عضو پشته کمکی مقایسه می‌کنیم. اگر کوچکتر بود، عضو جدید وگرنه آخرین عضو پشته‌ی کمکی را به پشته‌ی کمکی push می‌کنیم. از آنجایی که عضو انتهایی کمینه‌ی سایر اعضا تا به اینجا بوده‌است، با روشی که ارائه کردیم، عضو جدید پشته‌ی کمکی، کمینه‌ی اعضای پشته‌ی اصلی می‌باشد. برای عملیات پیدا کردن ماکسیمم نیز یک استک اضافه به همین شکل میگیریم. برای deleteMin، نمی‌توان این کار را انجام داد. برای اثبات فرض کنید که بشود deleteMin را در مرتبه‌ی $O(1)$ انجام داد. در این صورت تمام اعضای یک ارایه را در $O(n)$ درون این داده ساختار می‌ریزیم سپس یکی یکی کمینه را یافته در یک آرایه‌ی جدید ریخته و کمینه را از داده‌ساختار فعلی حذف کند تا داده ساختار خالی شود و هر کدام از این عملیات‌ها در $O(1)$ و در کل در $O(n)$ انجام می‌شود. در این صورت توانسته‌ایم که یک آرایه‌ی دلخواه را در زمان $O(n)$ مرتب کنیم که طبق قضایا می‌دانیم که مرتب‌سازی آرایه حداقل از مرتبه‌ی زمانی $\Omega(n \log(n))$ می‌باشد و این تناقض می‌باشد.

سؤال ۲. فرض کنید n عدد به صورت لیست پیوندی به شما داده شده است. با کمک گرفتن از مرتب‌سازی ادغامی و پیچیدگی زمانی $O(n \log n)$، اعداد را در شرایط زیر مرتب کنید.

آ. استفاده از حافظه اضافی از مرتبه $O(\log n)$

ب. استفاده از حافظه اضافی از مرتبه $O(1)$ [۱]

[۱] دقت کنید که استفاده از رابطه بازگشتی که تا عمق h پایین می‌رود، به اندازه $O(h)$ حافظه مصرف می‌کند. همچنین این سوال به صورت اضافی مطرح شده است و در کوئیز نمی‌آید. اما حل آن توصیه می‌شود.

پاسخ:

با استفاده از سه تابع زیر می‌توان لیست پیوندی را مرتب کرد :

```
head ← fisrt element of linked list
*head ← pointer to head
procedure merge_sort(*head)
    if head = null || head → next = null then
        return
    Split(head, &first_half, &second_half)
    merge_sort(&first_half)
    merge_sort(&second_half)
    *head = merge(&first_half, &second_half)
procedure split(*head, *first_half, *second_half)
    slow = head
    fast = head → next
    while fast ≠ null do
        fast = fast → next
        if fast ≠ null then
            slow = slow → next
            fast = fast → next
    *first_half = head
    *second_half = slow → next
procedure merge(*first_half, *second_half)
    if first_half = null then
        return second_half
    if second_half = null then
        return first_half
    if first_half ≤ second_half then
        result = first_half
        result → next = merge(first_half, second_half → next)
    else
        result = second_half
        result → next = merge(first_half, second_half → next, )
    return result
```

قسمت ۲:

لینک حل

```c
typedef struct _aList {
    struct _aList* next;
    struct _aList* prev; // Optional.
    // some data
} aList;

aList *merge_sort_list(aList *list,int (*compare)(aList *one,aList *two))
{
    int listSize=1,numMerges,leftSize,rightSize;
    aList *tail,*left,*right,*next;
    if (!list || !list->next) return list;  // Trivial case

    do { // For each power of two<=list length
        numMerges=0,left=list;tail=list=0; // Start at the start

        while (left) { // Do this list_len/listSize times:
            numMerges++,right=left,leftSize=0,rightSize=listSize;
            // Cut list into two halves (but don't overrun)
            while (right && leftSize<listSize) leftSize++,right=right->next;
            // Run through the lists appending onto what we have so far.
            while (leftSize>0 || (rightSize>0 && right)) {
                // Left empty, take right OR Right empty, take left, OR compare.
                if (!leftSize)                  {next=right;right=right->next;rightSize--;}
                else if (!rightSize || !right)  {next=left;left=left->next;leftSize--;}
                else if (compare(left,right)<0) {next=left;left=left->next;leftSize--;}
                else                            {next=right;right=right->next;rightSize--;}
                // Update pointers to keep track of where we are:
                if (tail) tail->next=next;  else list=next;
                // Sort prev pointer
                next->prev=tail; // Optional.
                tail=next;
            }
            // Right is now AFTER the list we just sorted, so start the next sort there.
            left=right;
        }
        // Terminate the list, double the list-sort size.
        tail->next=0,listSize<<=1;
    } while (numMerges>1); // If we only did one merge, then we just sorted the whole list.
```

```
39      return list;
40  }
```

سؤال ۳. به سؤالات زیر در مورد مرتب‌سازی درجی پاسخ دهید.

آ. در یک آرایه یک‌نابه‌جایی برابر یک جفت جایگاه است که عدد داخل جایگاه اولی بزرگتر از عدد داخل جایگاه دومی باشد. برای مثال در یک آرایه مرتب‌شده تعداد نابه‌جایی‌ها برابر صفر است و در یک آرایه مرتب‌شده برعکس این تعداد برابر $\binom{n}{2}$ است.
حال ثابت کنید اگر تعداد نابه‌جایی های یک آرایه dباشد. و الگوریتم insertion sort را روی آن اجرا کنیم با مرتبه $O(n+d)$ مرتب‌سازی انجام می‌شود.

ب. حال فرض کنید یک آرایه نامرتب به ما داده‌اند به طوریکه هر کس با جای اصلی آن حداکثر ۱۰ جایگاه فاصله دارد.برای مثال عدد کمینه حداکثر در اندیس ۱۰ است. با ارائه روش مناسب برای مرتب‌سازی ثابت کنید $O(n)$ این کار ممکن است.

پاسخ: در Insertion sort یک حلقه n تایی داریم و به ازای هر اندیس به اندازه تعداد اعداد بزرگتر از خودش که قبل از او ظاهر شده‌اند پیمایش می‌کنیم که این تعداد دقیقا برابر نابه‌جایی هایی است که این اندیس می‌سازد. پس در مجموع به ازای هر اندیس این مقادیر را اگر جمع کنیم به ما مجموع تعداد نابه‌جایی ها را می‌دهد.

حالا برای بخش دوم ثابت می‌کنیم تعداد نابه‌جایی های دنباله‌ای با این شرط $O(n)$ است. برای اینکار صرفا کافیست بگوییم اگر عددی مثل $a > b$ قبل از b ظاهر شده باشد. $a - b \leq 20$ چرا که در غیر این‌صورت یکی از a یا b با جایگاه اصلی‌شان بیش از ۱۰ واحد فاصله دارند. بنابراین تعداد نابه‌جایی‌ها حداکثر $20n$ است و طبق بخش الف با اجرای مرتب‌سازی درجی الگوریتمی از $O(n + 20n) = O(n)$ داریم.

سؤال ۴. فرض کنید که یک ویرایشگر داریم که دستورهای زیر را میپذیرد.

- مکان‌نما [۲] را به راست ببرد.
- مکان‌نما را به چپ ببرد.
- عمل پسبرد [۳] انجام دهد.
- یک کاراکتر انگلیسی اضافه کند.

اگر k دستور به عنوان ورودی به ما داده شود، خروجی را با پیچیدگی زمانی $O(k)$ چاپ کنید.

پاسخ: برای نگه داری متن از یک لیست پیوندی و برای نگه داری مکان مکان نما، از یک pointer استفاده کنید. در این صورت برای هر یک از عملیات های جا بجایی pointer حداکثر مقدار ثابتی زمان و برای حذف کردن یا اضافه کردن یک حرف به داده ساختار حداکثر $4 \times k$ بار عوض کردن هایpointer لیست پیوندی داریم. پس در مجموع $O(k)$ عملیات احتیاج است.

---

[۲] Cursor
[۳] Backspace

سؤال ۵. فرض کنید که A یک آرایه به طول n از اعداد است. برای هر اندیس i اولین خانه سمت راستش که مقداری بزرگتر مساوی A_i دارد را در پیچیدگی زمانی $O(n)$ بیابید.

پاسخ: مرحله اول ـ اولین عنصر لیست را در استک پوش کنید
مرحله دوم ـ عنصر بعدی را next بنامید . next را با عنصر بالای استک مقایسه کنید . مادامی که next بزرگتر بود، عنصر بالای استک را pop کنید .
مرحله سوم ـ next را در استک پوش کنید.
مرحله چهارم ـ اگر که عنصر بعدی در آرایه موجود بود، به مرحله دوم بروید .
تحلیل الگوریتم:
هر عنصر یکبار در استک push و ماکزیمم یک بار pop می شود . هر مقایسه ای هم که انجام می شود یا متناظر است با یک عملیات pop یا این که باعث می شود که حلقه داخل الگوریتم پایان بپذیرد و عنصر بعدی داخل استک push شود . پس در کل الگوریتم ما از O(n) است .

سؤال ۶. در لیست L اعمال زیر تعریف شده اند:

$Insert(L, x)$: عنصر x را به انتهای L اضافه می‌کند.

$Delete(L)$: عنصر انتهایی L را حذف می کند.

$multi - Delete(L, k)$: عنصر انتهایی k L را حذف می‌کند.

اگر n تا از اعمال فوق به ترتیب صحیح و دلخواه روی لیست L که در ابتدا تهی است اعمال شود، هزینه هر عمل به صورت سرشکن چقدر خواهد بود؟

پاسخ:

دستورات Insert و Delete و Multi-Delete به ترتیب $O(1)$ ،$O(1)$و $O(k)$ هستند. در نظر بگیرید هر شیئی که در لیست Insert می‌شود، حداکثر یک بار می‌تواند Delete شود. پس تعداد Delete ها حداکثر برابر تعداد Insert ها است و بدیهی است که حداکثر تعداد Insert برابر با n است. بنابراین تعداد عملیات Delete هم حداکثر برابر با n است. اگر $T(n)$ را هزینه ی n عملیات در نظر بگیریم؛ داریم :

$$T(n) \leq 2n \rightarrow T(n) = O(n) \rightarrow \frac{T(n)}{n} = O(1)$$

پس هزینه ی n عملیات به صورت سرشکن $O(1)$ می‌شود.

سؤال ۷. به کمک قضیه اصلی توابع بازگشتی زیر را تحلیل کنید. در صورتی که تابعی با قضیه اصلی قابل تحلیل نباشد، فقط دلیل آن را ذکر کنید (نیازی به تحلیل این توابع نیست).

آ. $T(n) = 9T(\frac{n}{3}) + n^2$

ب. $T(n) = 3T(\frac{n}{5}) + n^{0.6}$

پ. $T(n) = 3T(\frac{n}{3}) + \frac{n}{\log n}$

ت. $T(n) = 16T(\frac{n}{4}) + n!$

ث. $T(n) = \frac{1}{4}T(\frac{n}{2}) + \frac{1}{n}$

ج. $T(n) = 2T(\sqrt[3]{n}) + (\log n)^2$ [۴]

پاسخ:

چ. $\log_3 9 = 2$ و داریم $f(n) = n^2 = \Theta(n^2)$ پس نتیجه میگیریم که $T(n) = \Theta(n^2 \log n)$ (حالت دوم از قضیه اصلی)

ح. $n^{\log_b a} = n^{0.682}$ و چون $n^{0.6}$ از $n^{0.682}$ به صورت چند جمله ای کوچکتر است پس در این حالت جواب $\theta(n^{0.682})$ است. (حالت اول قضیه اصلی)

خ. چون که $\log_b a = 1$ و اینکه n^1 از $\frac{n}{\log n}$ به صورت چند جمله‌ای کوچکتر نیست، پس نمی‌توان از قضیه اصلی استفاده کرد.

د. $\log_4 16 = 2$ و داریم $f(n) = n! = \Omega(n^{2+\epsilon})$ پس نتیجه میگیریم که $T(n) = \Theta(n!)$ (حالت سوم از قضیه اصلی)

ذ. چونکه $\log_2 \frac{1}{4} = -2$ و اینکه $\frac{1}{n}$ از $\frac{1}{n^2}$ به صورت چند جمله‌ای کوچکتر نیست، پس نمی‌توان از قضیه اصلی استفاده کرد.

ر. از تغییر متغیر $m = \log n$ استفاده می‌کنیم و سپس قرار می‌دهیم $T(2^m) = S(m)$.
پس داریم: $S(m) = 2S(\frac{m}{3}) + m^2$ در این‌جا از قضیه اصلی استفاده می‌کنیم و بدست میاوریم $S \in \Theta(m^2)$ پس $T \in \Theta(\log(m)^2$

سؤال ۸. روابط بازگشتی زیر را تحلیل کنید.

آ. $T(n) = \sqrt{n} \times T(\sqrt{n}) + n$

ب. $T(n) = T(\frac{n}{2} + \sqrt{n}) + \sqrt{6046}$

پ. $T(n) = mT(\frac{n}{m}) + O(n)$ [۵]
($m > 1$ و n توانی از m)

ت. $T(n) = T(\lceil \frac{n}{2} \rceil) + T(\lfloor \frac{n}{2} \rfloor) + O(n)$ [۶]

پاسخ: قسمت ۱: تعریف می کنیم $f(n) = \frac{T(n)}{n}$ حال با جایگزینی در صورت اصلی رابطه داریم $f(n) = f(\sqrt{n}) + 1$ حال به استقرا اثبات می کنیم که $\forall k : f(n) = f(n^{\frac{1}{2^k}}) + k$ به وضوح حکم برای ۱ برقرار است حال اگر حکم برای $1, 2, .., k$ برقرار باشد $f(n) = f(n^{\frac{1}{2^{k+1}}}) + k + 1$ $f(n) = f(n^{\frac{1}{2^k}}) + k$ $f(f(n^{\frac{1}{2^k}}) = f(n^{\frac{1}{2^{k+1}}}) + 1$ $\Rightarrow$ و حکم استقرا ثابت می شود. حال اگر در صورت تعمیم یافته رابطه قرار دهیم $k = log(log(n))$ داریم $f(n) = f(2) + log(log(n))$ حال با توجه به نحوه تعریف داریم $T(n) = \theta(\sqrt{n} \times log(log(n))$

---

[۴] راهنمایی : از تغییر متغیر $m = \log n$ استفاده کنید.
[۵] با استفاده از روش استقرا این قسمت را تحلیل کنید.
[۶] n .لزوماً توانی از ۲ نیست

قسمت ۲: داریم $\exists N \forall n > N : \frac{n}{2} < \frac{n}{2} + \sqrt{n} < \frac{2n}{3}$ و همچنین با استفاده از استقرا می توان ثابت کرد که T صعودی است حال داریم $T(n) < c_1 \times log(n)\ T(n) < T(\frac{2n}{3}) + \sqrt{6046})) < ... < T(\frac{2^k n}{3^k}) + k \times \sqrt{6046}$ و از طرفی دیگر $\Rightarrow T(n) = \theta(log(n)) \Rightarrow T(n) > c_2 \times log(n)\ T(n) > T(\frac{n}{2}) + \sqrt{6046})) > ... > T(\frac{n}{2^k}) + k \times \sqrt{6046}$

قسمت ۳: ابتدا جملات کوچک تر را بررسی می‌کنیم تا به یک الگوی کلی دست پیدا کنیم.

$$n = m \rightarrow T(m) = mT(1) + O(m) = mO(1) + O(m) = O(m) + O(m) = 2O(m)$$

$$n = m^2 \rightarrow T(m^2) = mT(m) + O(m^2) = 2mO(m) = O(m^2) = 2O(m^2) + O(m^2) = 3O(m^2)$$

پس متوجه میشویم $T(m^k)$ از الگوی زیر پیروی می کند.

$$T(m^k) = (k+1)O(m^k)$$

حال از استقرا استفاده می‌کنیم. $T(n) = mT(\frac{n}{m}) + O(n)$

$$T(m^k) = mT(m^{k-1}) + O(m^k) = mkO(m^{k-1}) + O(m^k) = kO(m^k) + O(m^k) = (k+1)O(m^k)$$

حال رابطه ی نهایی را بر حسب n می‌نویسیم.

$$T(n) = (log_m^n + 1)O(n) = O(nlogn)$$

قسمت ۴: رابطه ی بازگشتی نوشته شده در این قسمت همان sort merge است. برای اثبات این قسمت هم از استقرا استفاده می‌کنیم.

$$T(n) \leq T(\lceil \frac{n}{2} \rceil) + T(\lfloor \frac{n}{2} \rfloor) + c_2 n$$

$$T(1) = c_1$$

$$T(n) \leq c_3 nlog(n)$$

در ابتدا تعریف می‌کنیم که $c_3 = 2c_2 + c_1$

پایه استقرا:

$$T(2) \leq T(1) + T(1) + 2c_2 \leq c_1 + c_1 + 2c_2 \leq c_1 + c_3 \leq 2c_3 \leq c_3 2log(2)$$

فرض استقرا

$$T(n) \leq T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + c_2 n \leq c_3 \lfloor \frac{n}{2} \rfloor log \lfloor \frac{n}{2} \rfloor\ c_3 \lceil \frac{n}{2} \rceil log \lceil \frac{n}{2} \rceil + c_2 n \leq c_3 \lfloor \frac{n}{2} \rfloor log \lceil \frac{n}{2} \rceil\ c_3 \lceil \frac{n}{2} \rceil log \lceil \frac{n}{2} \rceil + c_2 n \leq c_3 (\lfloor \frac{n}{2} \rfloor + \lceil \frac{n}{2} \rceil) log \lceil \frac{n}{2} \rceil + c_2 n \leq c_3 nlog \lceil \frac{n}{2} \rceil + c_2 n \leq c_3 nlog \lceil \frac{n}{2} \rceil + \frac{c_3}{2} n \leq c_3 n(log \lceil \frac{n}{2} \rceil + \frac{1}{2}) \leq c_3 n(log \lceil \frac{n}{2} \rceil \sqrt{2}) \leq c_3 nlog(n)$$

موفق باشید